/Tratamento De Exceções/

Flora Belas

/O que é/

- O **tratamento de exceções** em Java é o mecanismo que permite gerenciar erros e situações imprevistas durante a execução do programa, evitando que ele termine abruptamente e mantendo o fluxo normal de processamento. Quando ocorre uma exceção, o código é interrompido e direcionado para um manipulador de exceção (exception handler), que define como recuperar a situação de erro.

/O que é/

- Quando algo dá errado, o Java faz cria um objeto que representa aquele erro, com informações sobre o que aconteceu, e "joga" esse objeto pra cima. O termo técnico é lançar (throw). Se ninguém pegar esse objeto no caminho, o programa encerra e imprime um texto vermelho no console.
- O texto é o Java entregando um relatório. Ele diz o nome da exceção (que descreve o tipo de problema), a mensagem (que descreve o caso específico) e o caminho até a linha onde quebrou.
- Tratar uma exceção significa pegar esse objeto antes que ele derrube o programa, decidir o que fazer, e seguir em frente. É isso que o try/catch faz.

/Exception Handler/

- Um exception handler (manipulador de exceção) é um bloco de código responsável por detectar e responder a esses eventos anômalos que interrompem o fluxo normal da execução de um programa.
- Sua função principal é garantir que a aplicação reaja de forma controlada a erros, como falhas de rede, entradas inválidas ou divisões por zero, evitando que o sistema trave ou exiba comportamentos inesperados.
- Quando uma exceção é lançada, o sistema de execução busca o primeiro manipulador compatível para processar o erro. Isso permite que o desenvolvedor crie ações específicas, como exibir uma mensagem, registrar detalhes técnicos para depuração ou recuperar o estado anterior da aplicação.

/Ler uma Exceção/

```
Escreva seu número:
a
Exception in thread "main" java.lang.NumberFormatException Create breakpoint : For input string: "a"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:665)
    at java.base/java.lang.Integer.parseInt(Integer.java:781)
    at org.example.aula10.TratamentoExcecao.main(TratamentoExcecao.java:12)

Process finished with exit code 1
```

Nome da exceção

A mensagem explicando o erro

Em que arquivo seu aconteceu

Em que linha aconteceu

/Tipos de erro/

Tipo	Quando acontece	Exemplo
Erro de compilação (sintaxe)	O código nem compila	Esquecer ;, usar variável inexistente
Erro em tempo de execução (runtime)	O programa compila, mas quebra ao rodar	Divisão por zero, acessar índice fora do array
Erro lógico	O programa roda "perfeito", mas o resultado está errado	Subtrair em vez de somar

/Error vs Exception/

- Error → A máquina quebrou, não tem o que fazer.
- Exception → Algo inesperado aconteceu no código, dá pra lidar.

	Error	Exception
Causa	Falta de recursos do sistema (memória, stack)	Problema no código da aplicação
Recuperação	Quase impossível	Possível com try-catch
Exemplos	OutOfMemoryError, StackOverflowError	ArithmeticException, IOException
Tratar?	Na prática, não, o programa morre	Sim — é pra isso que existe o handler

/Checked vs Unchecked/

Checked é um tipo de exception que o compilador te obriga a tratar. Se não usar try-catch nem throws, o código não compila. Você chama o comando e se não adicionar um try catch, não compila.

O compilador não liga. O código compila sem problema, mas pode quebrar na hora de rodar.

/Checked vs Unchecked/

	Checked	Unchecked
Quem avisa?	O compilador (obriga tratar)	Ninguém, quebra na runtime
Causa típica	Fator externo (arquivo, rede, banco)	Bug no seu código (null, índice inválido)
Exemplo	IOException	NullPointerException

/try catch/

O **try** é um bloco onde você coloca o código que pode dar errado. É você dizendo ao Java: "tenta executar isso aqui, e se algo der problema, não derruba o programa, me avisa".

O catch é o plano B. Ele só executa se uma exceção acontecer dentro do try.

/try catch/

O que acontece na prática: o Java executa o try linha por linha normalmente. Se nada der errado, ele pula o catch inteiro e segue a vida. Se algo der errado, ele abandona o try na hora — as linhas seguintes do try não rodam — e salta direto pro catch.

Depois que o catch termina, o programa continua normalmente a partir dali. É essa a diferença: sem tratamento o programa morre; com tratamento ele lida com o problema e segue.

/finally/

- O bloco finally roda **sempre**
- Deu certo? Roda. Deu erro? Roda também.
- Vem depois do catch
- É **opcional**. Nem todo try/catch precisa
- Serve pra "fazer a faxina“. Fechar arquivo, fechar conexão, encerrar recurso, coisas que precisam acontecer de qualquer jeito

/try catch finally/

```
try {
    int resultado = 10 / 0;
    System.out.println(resultado);

} catch (ArithmeticException e) {
    System.out.println("Não dá pra dividir por zero!");

} finally {
    System.out.println("Isso sempre roda.");
}

System.out.println("O programa continua.");
```